

PLANO DE TESTES — ISO/IEC/IEEE 29119

Projeto: App de Delivery

Tecnologia: Flutter / Dart

Data: 12/05/2026

Documento A — Base Conceitual de Teste

Norma aplicada: ISO/IEC/IEEE 29119-1

1. Sistema sob Teste

O sistema sob teste compreende o frontend do aplicativo de delivery desenvolvido em Flutter. O escopo de teste é limitado às unidades testáveis de forma isolada, sem dependência de conexão real com serviços externos.

2. Itens de Teste

- ProductModel: modelo de dados responsável pela serialização e deserialização de produtos.
- ProductService: serviço HTTP responsável pela comunicação com o endpoint /produto do backend.

3. Escopo

- Deserialização de JSON (fromJson) com diferentes tipos e formatos de campo.
- Serialização para mapa local (toMap / fromMap).
- Serialização para envio ao backend (toBackendMap).
- Comunicação HTTP do serviço ProductService (getAll).
- Tratamento de respostas de sucesso e de erro.

4. Fora de Escopo

- Firebase Firestore e Firebase Auth.
- API de produção (requisições reais).
- Backend Node.js (Carrinho, Pedido, Produto, User).
- Testes de interface (widgets e telas).
- Performance e segurança.

5. Requisitos Funcionais Cobertos

ID	Requisito
RF01	O sistema deve deserializar corretamente um produto com imagem no formato Map (Cloudinary).
RF02	O sistema deve deserializar corretamente um produto com imagem no formato String.
RF03	O sistema deve tratar imagem nula e retornar string vazia.
RF04	O sistema deve converter o campo valor para double independente do tipo recebido.
RF05	O sistema deve tratar valor inválido e retornar 0.0.
RF06	O sistema deve converter id numérico para String.
RF07	O sistema deve aceitar categoria nula sem falha.
RF08	O sistema deve reconstruir corretamente um modelo a partir de toMap().
RF09	O sistema deve gerar mapa backend com chaves em português.
RF10	O sistema deve retornar lista de produtos ao receber status 200.
RF11	O sistema deve lançar exceção ao receber status 404 ou 500.
RF12	O sistema deve retornar lista vazia ao receber body com data vazio.

6. Condições de Teste

Cond.	Descrição
CT01	Validar fromJson com imagem como Map (Cloudinary)
CT02	Validar fromJson com imagem como String
CT03	Validar fromJson com imagem null
CT04	Validar fromJson com valor como int
CT05	Validar fromJson com valor como double
CT06	Validar fromJson com valor como String numérica

Cond.	Descrição
CT07	Validar fromJson com valor inválido
CT08	Validar fromJson com id numérico
CT09	Validar fromJson com categoria ausente
CT10	Validar toMap e fromMap (round-trip)
CT11	Validar toBackendMap com chaves corretas
CT12	Validar toBackendMap com categoria null
CT13	Validar fromJson com campos nome e descricao ausentes
CT14	Validar getAll com status 200
CT15	Validar getAll com status 404
CT16	Validar getAll com status 500
CT17	Validar getAll com lista vazia
CT18	Validar getAll acessa URL correta

7. Tipos de Teste

— Teste de Unidade: valida cada método isoladamente, sem dependências externas reais.

8. Riscos

ID	Risco
R01	Alteração no contrato de resposta da API quebrar a deserialização silenciosamente.
R02	Campo imagem receber formato inesperado e retornar string vazia sem aviso.
R03	Campo valor receber null e não ser tratado, causando exceção em runtime.
R04	ProductService não lançar exceção em caso de erro HTTP, permitindo uso de dados inválidos.
R05	toBackendMap enviar chaves incorretas ao backend, causando falha na criação do pedido.

Documento B — Processo de Teste

Norma aplicada: ISO/IEC/IEEE 29119-2

1. Estratégia de Teste

Serão aplicados exclusivamente testes de unidade, utilizando o pacote `flutter_test` nativo do Flutter SDK. As dependências externas (HTTP) serão substituídas por mocks via pacote `http` com `MockClient` disponível em `package:http/testing.dart`.

2. Ambiente de Teste

Componente	Detalhe
Flutter SDK	Versão compatível com o projeto
Dart SDK	Incluído no Flutter SDK
<code>flutter_test</code>	Pacote nativo — nenhuma instalação adicional
<code>package:http/testing.dart</code>	<code>MockClient</code> para simular respostas HTTP
IDE	VS Code ou Android Studio

3. Critérios de Entrada

- Projeto Flutter funcional e compilando sem erros.
- `ProductModel` e `ProductService` implementados.
- Documento A concluído e revisado.

4. Critérios de Saída

- Todos os 19 casos de teste executados.
- Resultados registrados na tabela de controle (seção 6).
- Nenhum teste crítico reprovado sem justificativa documentada.

5. Ordem de Execução

1. Testes de `ProductModel` (CT01 a CT13) — sem dependência de mock.
2. Testes de `ProductService` (CT14 a CT18) — requerem `MockClient` configurado.

6. Estrutura de Arquivos

A estrutura de testes deve seguir o padrão da pasta `test/` do projeto Flutter:

```
test/  
  unit/  
    product_model_test.dart  
    product_service_test.dart
```

7. Tabela de Controle

ID	Caso de Teste	Status
TC01	fromJson — imagem como Map (Cloudinary)	Planejado
TC02	fromJson — imagem como String	Planejado
TC03	fromJson — imagem null	Planejado
TC04	fromJson — valor como int	Planejado
TC05	fromJson — valor como double	Planejado
TC06	fromJson — valor como String numérica	Planejado
TC07	fromJson — valor inválido	Planejado
TC08	fromJson — id como int	Planejado
TC09	fromJson — categoria ausente	Planejado
TC10	toMap e fromMap — round-trip	Planejado
TC11	toBackendMap — chaves em português	Planejado
TC12	toBackendMap — categoria null	Planejado
TC13	fromJson — nome e descricao ausentes	Planejado
TC14	getAll — status 200	Planejado
TC15	getAll — status 404	Planejado
TC16	getAll — status 500	Planejado
TC17	getAll — lista vazia	Planejado
TC18	getAll — URL correta	Planejado

8. Execução

Para executar os testes de unidade, utilizar o comando:

flutter test

9. Conclusão

O processo de teste será encerrado após a execução completa de todos os casos planejados e o registro de resultados na tabela de controle acima.

Documento C — Técnicas e Casos de Teste

Norma aplicada: ISO/IEC/IEEE 29119-4

1. Técnicas Utilizadas

Técnica	Finalidade
---------	------------

Técnica	Finalidade
Particionamento de Equivalência	Separar entradas válidas e inválidas para cada campo.
Valor Limite	Validar comportamento em entradas no limite (null, string vazia, 0).
Transição de Estado	Validar que o modelo transita corretamente entre formatos de dados.
Teste Baseado em Cenário	Validar o fluxo completo de chamada HTTP e mapeamento de resposta.

2. Derivação dos Casos de Teste

2.1 ProductModel — fromJson, toMap, fromMap, toBackendMap

ID	Condição	Técnica	Entradas / Condição	Resultado Esperado
TC01	CT01 — imagem como Map	Particionamento	imagem: {url: 'https://img.jpg'}	image == 'https://img.jpg'
TC02	CT02 — imagem como String	Particionamento	imagem: 'https://img.jpg'	image == 'https://img.jpg'
TC03	CT03 — imagem null	Valor Limite	imagem: null	image == ''
TC04	CT04 — valor como int	Particionamento	valor: 25	price == 25.0
TC05	CT05 — valor como double	Particionamento	valor: 19.99	price == 19.99
TC06	CT06 — valor String numérica	Particionamento	valor: '29.90'	price == 29.90
TC07	CT07 — valor inválido	Valor Limite	valor: 'abc'	price == 0.0
TC08	CT08 — id como int	Transição de Estado	id: 42 (int)	id == '42' (String)
TC09	CT09 — categoria ausente	Valor Limite	sem campo 'categoria'	categoria == null
TC10	CT10 — round-trip toMap/fromMap	Transição de Estado	model → toMap() → fromMap()	model reconstruído igual ao original
TC11	CT11 — toBackendMap chaves	Particionamento	ProductModel instanciado	mapa contém: produtoId, nome, descricao, valor,

ID	Condição	Técnica	Entradas / Condição	Resultado Esperado
				imagem, categoria
TC12	CT12 — toBackendMap categoria null	Valor Limite	categoria == null	campo 'categoria' == "
TC13	CT13 — campos ausentes	Valor Limite	sem 'nome' e sem 'descricao'	name == " e description == "

2.2 ProductService — getAll()

ID	Condição	Técnica	Entradas / Condição	Resultado Esperado
TC14	CT14 — status 200	Cenário	MockClient retorna 200 + JSON com 2 produtos	Lista com 2 ProductModel mapeados
TC15	CT15 — status 404	Particionamento	MockClient retorna 404	Lança Exception('Falha ao carregar produtos')
TC16	CT16 — status 500	Particionamento	MockClient retorna 500	Lança Exception('Falha ao carregar produtos')
TC17	CT17 — lista vazia	Valor Limite	MockClient retorna 200 + {data: []}	Retorna lista vazia sem erro
TC18	CT18 — URL correta	Cenário	MockClient configurado	URL chamada contém '/produto'

3. Tabela Consolidada de Técnicas

Condição	Técnica Aplicada
CT01	Particionamento de Equivalência
CT02	Particionamento de Equivalência
CT03	Valor Limite
CT04	Particionamento de Equivalência
CT05	Particionamento de Equivalência
CT06	Particionamento de Equivalência
CT07	Valor Limite

Condição	Técnica Aplicada
CT08	Transição de Estado
CT09	Valor Limite
CT10	Transição de Estado
CT11	Particionamento de Equivalência
CT12	Valor Limite
CT13	Valor Limite
CT14	Teste Baseado em Cenário
CT15	Particionamento de Equivalência
CT16	Particionamento de Equivalência
CT17	Valor Limite
CT18	Teste Baseado em Cenário

4. Conclusão

As condições de teste identificadas no Documento A foram derivadas em 19 casos de teste formais utilizando as técnicas definidas pela norma ISO/IEC/IEEE 29119-4. Os casos estão preparados para implementação automatizada na pasta test/ do projeto Flutter, utilizando o pacote flutter_test e MockClient para simulação de respostas HTTP.